

TP1 - Prise en main de Yocto Project

1. Environnement de travail

La machine de compilation possède les caractéristiques suivantes :

- système hôte : Ubuntu 26.04 LTS x86-64 ;
- processeurs disponibles : 16 ;
- espace disque total : environ 145 Go ;
- branche Yocto/Poky : `scarthgap` ;
- version de Poky : 5.0.19 ;
- version de BitBake : 2.8.1 ;
- machine cible : `qemux86-64` .

La connexion à la machine étant réalisée en SSH, une session `tmux` a été utilisée. Elle permet de laisser la compilation continuer même en cas de déconnexion :

```
tmux new -s tp0
```

Pour reprendre la session après une coupure :

```
tmux attach -t tp0
```

2. Préparation de l'hôte

Les dépendances nécessaires à Yocto ont été installées avec la commande suivante :

```
sudo apt update
sudo apt install -y \
gawk wget git diffstat unzip texinfo gcc build-essential \
chrpath socat cpio python3 python3-pip python3-pexpect xz-u
```

```
tils \  
debianutils iputils-ping python3-git python3-jinja2 libegl1  
\  
libssl1.2-dev pylint xterm python3-subunit mesa-common-dev  
\  
zstd lz4
```

Le support de cours est prévu pour Ubuntu 22.04. Deux paquets ont changé de nom dans Ubuntu 26.04 : `libegl1-mesa` a été remplacé par `libegl1`, et `liblz4-tool` par `lz4`.

Les versions principales installées sont :

```
Git 2.53.0  
GCC 15.2.0  
Python 3.14.4  
LZ4 1.10.0
```

3. Installation de Poky

Poky a été récupéré sur la branche LTS `scarthgap` :

```
cd ~  
git clone -b scarthgap https://git.yoctoproject.org/poky  
cd ~/poky  
git branch --show-current
```

La dernière commande a confirmé l'utilisation de la bonne branche :

```
scarthgap
```

L'utilisation d'une branche LTS permet de conserver un ensemble stable et cohérent de recettes. La branche `master`, qui évolue continuellement, n'est pas adaptée à ce type de travail reproductible.

4. Création de l'environnement de build

Le répertoire de compilation du TP a été initialisé avec :

```
cd ~/poky
source oe-init-build-env build-tp0
```

Cette commande a créé le répertoire suivant et y a automatiquement placé le terminal :

```
/home/louis/poky/build-tp0
```

Le script doit être exécuté avec `source`, car il modifie l'environnement du shell courant et rend notamment disponibles les commandes `bitbake`, `runqemu` et `bitbake-layers`.

5. Configuration de `local.conf`

La machine cible par défaut est :

```
MACHINE ??= "qemux86-64"
```

Les téléchargements et le cache de compilation sont conservés dans :

```
DL_DIR ?= "${TOPDIR}/downloads"
SSTATE_DIR ?= "${TOPDIR}/sstate-cache"
```

La machine hôte dispose de 16 processeurs. Le build a été limité à 8 tâches parallèles afin de conserver une marge de mémoire :

```
BB_NUMBER_THREADS = "8"
PARALLEL_MAKE = "-j 8"
```

`BB_NUMBER_THREADS` contrôle le nombre de tâches BitBake pouvant s'exécuter simultanément. `PARALLEL_MAKE` contrôle le parallélisme utilisé pendant les compilations basées sur Make.

6. Première compilation

La première image a été construite avec :

```
bitbake -k core-image-minimal
```

L'option `-k`, signifiant *keep going*, permet à BitBake de poursuivre les tâches indépendantes lorsqu'une erreur apparaît.

Deux problèmes liés à Ubuntu 26.04 ont dû être corrigés. La locale `en_US.UTF-8` a d'abord été générée :

```
sudo apt install -y locales
sudo locale-gen en_US.UTF-8
export LANG=en_US.utf8
export LC_ALL=en_US.utf8
```

Ubuntu restreint également les espaces de noms utilisateur non privilégiés avec AppArmor. BitBake en ayant besoin, cette restriction a été désactivée temporairement, jusqu'au prochain redémarrage :

```
echo 0 | sudo tee /proc/sys/kernel/apparmor_restrict_unpriv
ileged_usersns
```

Après ces corrections, les 4090 tâches de la première compilation se sont terminées avec succès. Un téléchargement direct de `ca-certificates` a échoué, mais BitBake a automatiquement utilisé un miroir alternatif. Aucun échec de tâche n'a été constaté.

7. Premier démarrage dans QEMU

La machine de compilation étant utilisée en SSH, l'image a été démarrée sans fenêtre graphique :

```
runqemu qemux86-64 nographic
```

La connexion à l'image se fait avec l'utilisateur `root`, sans mot de passe. L'arborescence du système a été vérifiée avec :

```
ls /
ls /usr/bin | head
```

Les trois outils demandés étaient absents de l'image initiale :

```
root@qemux86-64:~# htop
-sh: htop: not found

root@qemux86-64:~# strace
-sh: strace: not found

root@qemux86-64:~# tcpdump
-sh: tcpdump: not found
```

Ce comportement est normal. `core-image-minimal` ne contient que les composants indispensables au démarrage du système et à l'obtention d'un shell.

QEMU a ensuite été arrêté proprement avec :

```
poweroff
```

8. Personnalisation de l'image

Les trois paquets ont été ajoutés à la fin de `conf/local.conf` :

```
IMAGE_INSTALL:append = " htop strace tcpdump"
```

L'espace placé avant `htop` est indispensable. L'opérateur `:append` n'ajoute pas automatiquement de séparateur entre l'ancienne valeur et le texte ajouté.

9. Ajout des layers complémentaires

La première vérification a montré que Poky ne fournit pas directement toutes les recettes demandées. Le dépôt `meta-openembedded` a donc été cloné sur la même branche `scarthgap` :

```
cd ~/poky
git clone -b scarthgap https://git.openembedded.org/meta-op
embedded
cd ~/poky/build-tp0
```

La recette `htop` est fournie par le layer `meta-oe` :

```
bitbake-layers add-layer ../meta-openembedded/meta-oe
```

Sa présence a été confirmée avec :

```
bitbake-layers show-recipes htop
```

Résultat :

```
htop:
  meta-oe 3.3.0
```

La recette `tcpdump` se trouve dans `meta-networking`. Ce layer dépend de `meta-python` et de `meta-oe`. Les layers nécessaires ont donc été ajoutés dans cet ordre :

```
bitbake-layers add-layer ../meta-openembedded/meta-python
bitbake-layers add-layer ../meta-openembedded/meta-networking
```

Cette étape illustre le rôle des layers dans Yocto : ils permettent d'organiser les recettes par thème ou par fournisseur, puis de les combiner selon les besoins du produit.

10. Vérification avant reconstruction

La valeur finale de `IMAGE_INSTALL`, après interprétation de toute la configuration par BitBake, a été contrôlée avec :

```
bitbake -e core-image-minimal | grep '^IMAGE_INSTALL='
```

La ligne obtenue contient bien :

```
htop strace tcpdump
```

11. Seconde compilation et validation

L'image personnalisée a été reconstruite avec :

```
bitbake -k core-image-minimal
```

Cette seconde compilation est plus rapide que la première, car BitBake réutilise les résultats des tâches inchangées. Il compile principalement les nouveaux paquets, leurs dépendances, puis reconstruit l'image finale.

La nouvelle image est ensuite lancée dans QEMU :

```
runqemu qemux86-64 nographic
```

Les outils ajoutés sont vérifiés avec :

```
htop  
strace --version  
tcpdump --version  
strace ls /
```

`htop` affiche désormais la liste interactive des processus. `strace` permet d'observer les appels système effectués par une commande, tandis que `tcpdump` fournit les fonctions d'analyse du trafic réseau.

12. Rôle du `sstate-cache`

Le `sstate-cache`, ou cache d'état partagé, conserve les résultats réutilisables des tâches déjà effectuées par BitBake : compilation, installation, création de paquets et autres étapes intermédiaires. Lors de la seconde compilation, les composants inchangés ont été restaurés depuis ce cache. BitBake n'a donc reconstruit que les nouveaux paquets et les parties de l'image affectées par la modification. Ce mécanisme réduit fortement le temps nécessaire aux compilations successives.